

| Spec Driven Development

01

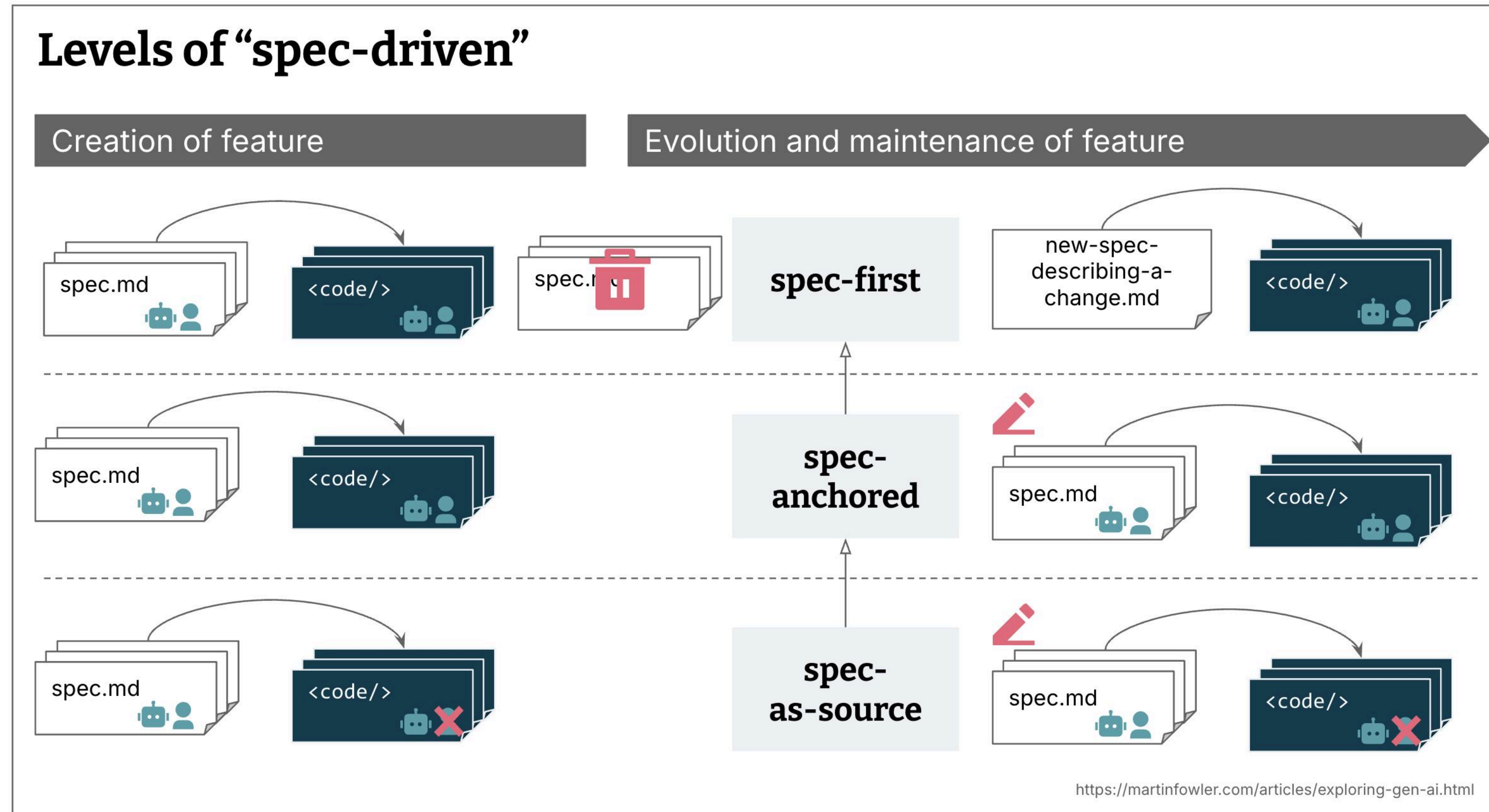
// Spec driven development - SDD

- Specification before code
- What problem are we solving? Who is affected? What are the limitations? What does success look like? What are we explicitly not building?
- Spec -> Plan -> Implement
- Specs intentionally should be human readable

Framework Name	Segmentation (Process Steps)	Tool Type	Model Binding	Costs
GitHub Spec Kit	Constitution → Specify → Plan → Tasks → Implement 1	CLI / Agent Extension	Agnostic (supports 8+ agents)	Free (MIT License)
OpenSpec	Proposal → Delta Specs → Design → Tasks → Implement → Verify → Archive	CLI / Agent Extension	Agnostic (supports 20+ agents)	Free (Open Source)
BMAD METHOD	Analysis → Planning → Solutioning → Implementation → Handoffs	CLI / IDE Presets 8	Agnostic (Claude, Cursor, Windsurf etc.)	Free (Open Source)
Amazon Kiro	Requirements → Design → Tasks → Execution 11	Standalone IDE 12	Bound (only Anthropic Claude)	Free Quota / 20\$ / 40\$ / 200 \$
Tessl	Define Intent → Structure Spec → Build	CLI / Tile System	Agnostic (works with various agents)	Registry Beta / Future Paid
Intent (Augment Code)	Analysis → Drafting → Delegation → Verification	Development Environment / Workspace	BYOA (supports 4+ agents)	From \$60/month (Credit-based)
GSD	Capture Idea → Research → Requirements → Roadmap → Execute → Verify	CLI (Terminal-based)	Agnostic	Free (Open Source)
Cursor (Composer)	Brainstorm → Spec → Plan → Implement	Standalone IDE	Agnostic (Claude, GPT, Gemini)	\$20/month (Pro Version)

01

// Spec driven development - SDD



Framework	Organized by	Spec stays alive?	Who is the source of truth?	Drift handling
GitHub Spec Kit	Feature / git branch (`001-...`)	• Snapshot per branch • Constitution document	The code	None
OpenSpec	Domain (`auth/`, `payments/`)	yes (delta merge on archive)	“The spec”	• Agentic (`verify`) • Not deterministic
BMAD METHOD	SDLC phase (PRD → Arch → Story)	Discarded once completed	The code	None

01

// SDD- The hard truth

*All frameworks preach "**spec as source of truth.**" Only OpenSpec builds the mechanics that make it "true".*

01

// SDD- The hard truth

- Most SDD frameworks have an enforcement problem
- Specs are passive - nothing checks whether code still matches
- Specs are just another abstraction from the implementation plan which is through away code
- We need enforcement through computational behaviour

That thing has a name: **a contract**

01

// Example - OpenSpec Requirements

- Given/When/Then Scenario
- Bind it to a test, and the vague spec becomes a contract
- That is the open frontier of SDD
- Apply Example Mapping to make it bullet proof

01

// Example - Gherkin BDD Tests with cucumber

▼  features

 session.feature

▼  steps

 session.steps.js

 cucumber.json

01

// Gherkin BDD Tests with cucumber

File: steps/session.steps.js

```
1  Given("a logged-in user", function () {
2    this.system.login();
3  });
4
5  When("{int} minutes pass without activity", function (minutes) {
6    this.system.advanceMinutes(minutes);
7  });
8
9  Then("the session is invalid", function () {
10   assert.strictEqual(this.system.sessionIsValid(), false);
11   });
12
13  Then("the session is still valid", function () {
14    assert.strictEqual(this.system.sessionIsValid(), true);
15  });
```

01

// Gherkin BDD Tests with cucumber

File: feature/session.feature

```
1 Feature: Session Managment
2 Scenario: Session expires after inactivity
3     Given a logged-in user
4     When 15 minutes pass without activity
5     Then the session is invalid
```

01

// Gherkin BDD Tests with cucumber

File: feature/session.feature

```
1   Feature: Session Managment
2   Scenario: Session expires after inactivity
3       Given a logged-in user
4       When 10 minutes pass without activity
5       Then the session is invalid
```



01

// Gherkin BDD Tests with cucumber

File: feature/session.feature

```
1 Feature: Session Managment
2 Scenario: Scenario: Locked account cannot
  log in
3   Given a locked user account
4   ...
```



Important: Mode set to **strict**

01

// What we get

- Example Mapping → the right rules and examples - correctness
- Gherkin Specs → Readable specs
- Cucumber → Framework for transforming specs into tests
- We can apply ATDD here as well - Gherkin Specs → Cucumber Tests → Code

// Working Slot

